

Тема: Вложенность функций в Python

1. Вложенный вызов (Nested Calls)

Это когда мы передаем результат работы одной функции в качестве «входа» для другой. Представь это как конвейер: деталь выходит из одного станка и тут же попадает в следующий.

Шаблон:

```
результат = внешняя_функция(внутренняя_функция(аргумент))
```

Пример:

```
def square(x):  
    return x * x  
  
def double(x):  
    return x * 2  
  
# Сначала выполнится square(3) = 9, затем double(9) = 18  
result = double(square(3))  
print(result) # Вывод: 18
```

2. Порядок выполнения («Принцип Матрешки»)

Программа всегда вычисляет выражения изнутри наружу. Пока внутренняя функция не даст ответ, внешняя не может начать работу.

Пример цепочки:

```
print(abs(round(-5.7)))  
# 1. round(-5.7) -> -6  
# 2. abs(-6) -> 6  
# 3. print(6)
```

3. Вложенное определение (Nested Functions)

Это создание функции внутри другой функции. Внутренняя функция становится «секретной» - она живет и работает только пока выполняется внешняя.

Зачем это нужно?

- Инкапсуляция: Скрыть логику, которая не нужна остальной программе.

- Доступ к переменным: Внутренняя функция видит всё, что происходит во внешней (это называется «замыканием»).

Шаблон кода:

```
def внешняя_функция(параметры):  
    # Код внешней функции  
  
    def внутренняя_функция(параметры):  
        # Код, доступный только внутри  
        return результат  
  
    # Мы вызываем внутреннюю функцию здесь же  
    return внутренняя_функция(данные)
```

Живой пример: Проверка скидки

```
def process_payment(amount):  
    def apply_discount(val):  
        return val * 0.9 # Скидка 10%  
  
    if amount > 1000:  
        amount = apply_discount(amount)  
  
    return f"Итого к оплате: {amount}"  
  
print(process_payment(1200)) # Выведет: Итого к оплате:  
1080.0  
# print(apply_discount(1200)) # ОШИБКА: функция не  
видна снаружи!
```

4. Реальные кейсы применения

Пример №1: Математические расчеты (Теорема Пифагора)

Вместо одной сложной формулы мы вызываем встроенную функцию корня `sqrt` и передаем ей результат сложения квадратов.

```
from math import sqrt

def get_hypotenuse(a, b):
    return sqrt(pow(a, 2) + pow(b, 2))

print(get_hypotenuse(3, 4)) # 5.0
```

Пример №2: Очистка данных пользователя

Часто данные из интернета приходят «грязными». Мы можем создать цепочку обработчиков.

```
def clean_string(text):
    return text.strip().lower()

name = "  ALIKHAN  "
print(f"Привет, {clean_string(name)}!") # Привет,
alikhan!
```

5. Плюсы и минусы (Коротко для запоминания)

Плюсы	Минусы
Чистота: Основной код не захламлен мелкими функциями.	Сложность: Если вложений больше 3-х, код трудно читать.
Безопасность: Внутреннюю функцию нельзя вызвать случайно.	Память: Каждый раз при вызове внешней функции создается новая внутренняя.
Порядок: Логика сгруппирована в одном месте.	Отладка: Тяжелее найти ошибку во вложенных слоях.

Практические задачи для закрепления

Задача 1: «Двойной расчет»

Напишите функцию `add_five(x)`, которая прибавляет 5, и функцию `multiply_ten(x)`, которая умножает на 10. Вызовите их в одной строке так, чтобы сначала к числу 2 прибавилось 5, а потом результат умножился на 10.

Задача 2: «Форматирование счета»

Напишите функцию `format_currency(value)`, которая добавляет знак "\$" перед числом. Напишите функцию `get_total(price, count)`, которая считает сумму и вызывает внутри `format_currency` для вывода результата.

Задача 3: «Защитник»

Создайте функцию `is_adult(age)`. Внутри создайте вложенную функцию `check_valid()`, которая проверяет, что `age` не отрицательное число. Если число отрицательное, функция должна вернуть "Ошибка", если положительное - вернуть результат проверки (`True/False`).

Задача 4: «Цепочка слов»

Используя встроенные функции `len()`, `str()` и `print()`, напишите вложенный вызов, который берет список `[1, 2, 3]`, считает его длину, превращает эту длину в строку и выводит текст: "Длина списка: 3".

Задача 5: «Матрешка»

Создайте функцию `outer()`. Внутри неё `inner_1()`, а внутри `inner_1()` - функцию `inner_2()`. Пусть самая глубокая функция возвращает "Я здесь!", а `outer()` выводит это на экран через цепочку вызовов.